

Manual

Installation Commands:

The solution can be run using a virtual environment of Python 3.11 with all the dependencies installed from the *requirements.txt* file:

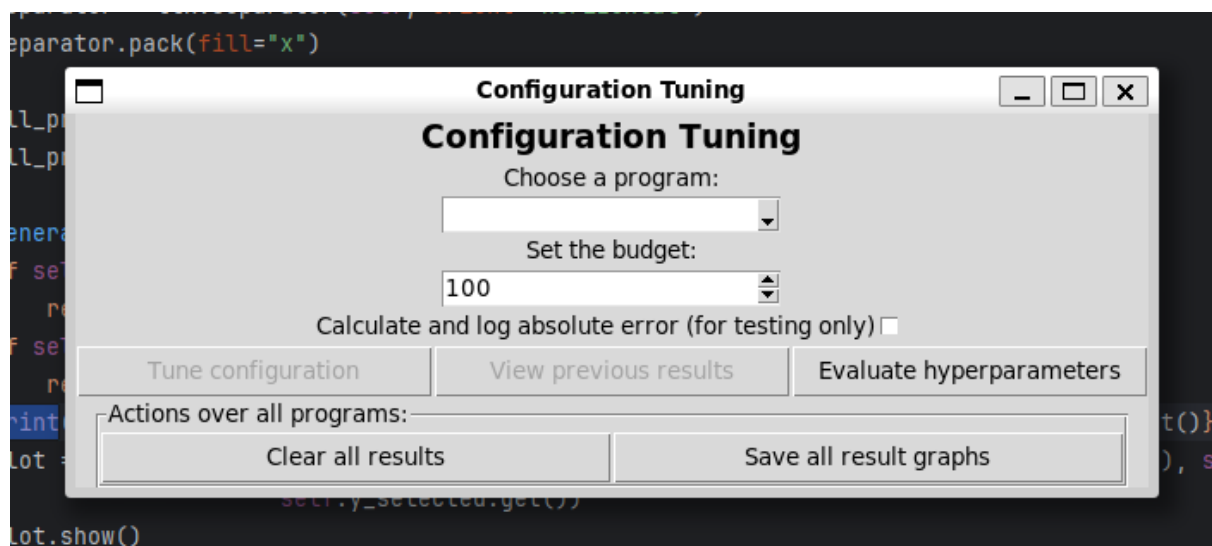
- `python3 -m venv venv`
- `source venv/bin/activate`
- `python3 -m pip install -r requirements.txt`

You may want to run `python3 -m pip install --upgrade pip` before the final command from above if the correct versions cannot be found.

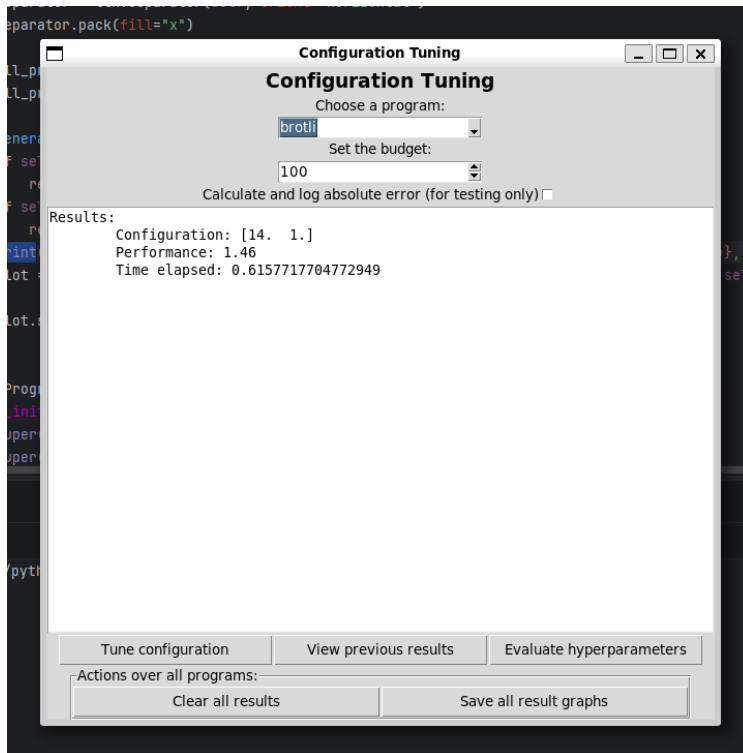
When running any of the files with a `.py` extension outlined in this document, enter the python virtual environment (if you are not in already) with “`source venv/bin/activate`” and then use the command “`python3 {filename}`” where {filename} is the name of the python file including its extension.

The Main Program:

Running *main.py* opens the UI interface for this program. Here there are options for running the tuning algorithm on a program. The “choose a program” combo box lists options of datasets of different configurations and objectives of programs that are saved in the “./datasets” directory.



Clicking the checkbox would force the program to do a brute force search when tuning the configuration to find the true optimal configuration and then will calculate the absolute error between its predicted optimal configuration and the true optimal configuration, with the error saved into the corresponding error file in the directory “./error”. To start the configuration tuning process “Tune Configuration” must be pressed. All tuning is a minimisation problem, unless changed manually in the code in “*configuration_tuning.py*” in the function “*tune_configuration_for()*” you could change the evaluation function in the call to “*tune_configuration()*” to “max” rather than “min”. The dataset that it is working on must have rows that are individual configurations in the rightmost columns and the single objective’s values in the rightmost column (or an absurdly large value if the objective value is not known). Outcomes of the tuning will be displayed and also logged in the corresponding file in “./results”.

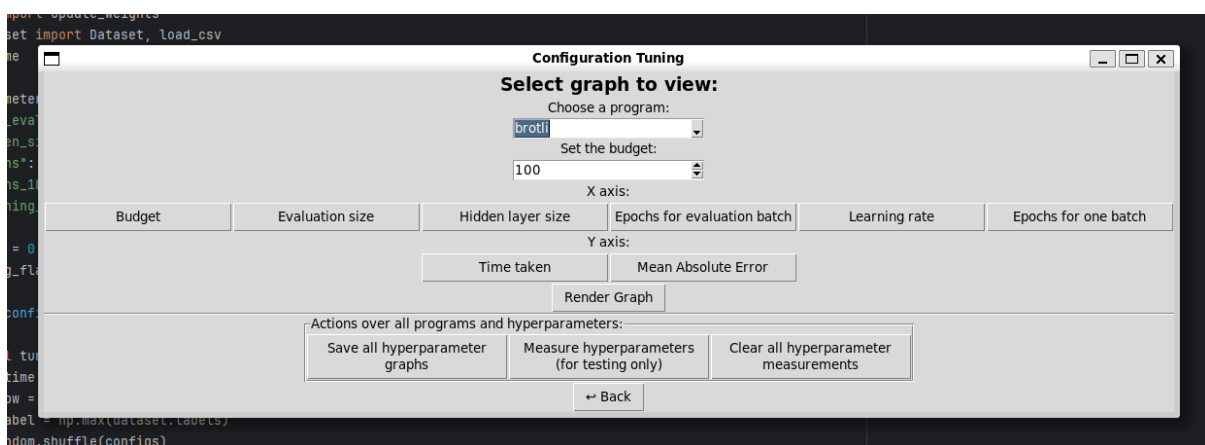


“Clear All Results” empties the “./results” folder.

“Save all results graphs” populates the “./graphs” folder with graphs of the optimal objective value calculated against iteration of the tuning algorithm for each program’s dataset. Similar graphs for each program in turn can be displayed when “View Previous Results” is pressed.

Clicking “Evaluate hyperparameters” brings up a different section of the program which is used to determine how different hyperparameters effect the error of the tuning program. Every option in this section except “Measure hyperparameters” uses

the default hyperparameters set at the top of “configuration_tuning.py”. Modify this to look at different values. Budget is not a hyperparameter of the tuning algorithm but is referred to as such in this document to simplify the language.



The top section of this page allows you to render graphs of your choice. When a hyperparameter is selected for the x axis, all other hyperparameters are kept constant at the values set in “configuration_tuning.py”.

Clicking “Save all hyperparameter graphs” populates the directory “./graphs” with all of the possible graphs that could be generated by this section of the program for all of the different programs’ datasets (there are 96 combinations for the 8 test datasets).

Generating these graphs uses the data saved in the directory “./error”, where hyperparameters are stored in rows for each iteration of the tuning algorithm for a certain program’s dataset, with the rightmost two columns being the values for time taken by the algorithm and the absolute error respectively. Clicking “Measure hyperparameters” will populate these files with a set range of each hyperparameter varied in turn for each programs’ dataset, with at least 90 iterations on

each hyperparameter. The fixed hyperparameters for this process are set in the file “measure_hyperparameter_performance.py” along with the budget.

You can clear the “./error” folder with the button “Clear all hyperparameter measurements”.

Other Actions:

Running “configuration_tuning.py” allows you to populate “./results”, running the tuning algorithm 100 times for each program’s dataset that is in “./datasets”. It uses the fixed hyperparameter values set at the top of this file, but it allows you to input the budget as an integer. It can take a while.

Running “dataset_builder.py” guides you in the creation of another csv in the directory “./datasets” for a new program. It generates all the possible combinations for multiple configuration options, but you will need to enter any measured objective values into the csv yourself.

Running “graph_visualise.py” saves histograms for all programs that have results in the “./results” directory into the “./graphs” directory. These histograms show the different tuned objectives split into bins to show which range the tuning algorithm most frequently gives.

Running “statistical_test.py” allows you to input the location of two different results files that will be compared using a Wilcoxon rank-sum test, returning a p-value for analysis. The idea is that you compare the output of this tuning algorithm for a specific program’s dataset to that of another tuning algorithm.